

IMPERIAL COLLEGE OF SCIENCE, TECHNOLOGY AND MEDICINE

EXAMINATIONS 2024

BEng Honours Degree in Computing Part II
MEng Honours Degrees in Computing Part II
BEng Honours Degree in Mathematics and Computer Science Part II
MEng Honours Degree in Mathematics and Computer Science Part II
for Internal Students of the Imperial College of Science, Technology and Medicine

*This paper is also taken for the relevant examinations for the
Associateship of the City and Guilds of London Institute*

PAPER COMP50004

OPERATING SYSTEMS

Wednesday 8th May 2024, 10:00

Duration: 90 minutes

Answer ALL TWO questions

Paper contains 2 questions
Calculators required

- 1 An operating system (OS) kernel runs on a multi-core machine and uses a preemptive scheduler. It executes two threads, T_1 and T_2 , which use a *semaphore* for protection of a critical region. Both threads execute the following code:

```
1 sem_t s;  
2 // ...  
3 sem_init(&s, X);  
4 // ...  
5 for (int i = 0; i < 2; i++) {  
6     sem_down(&s)  
7     critical_region(i);  
8     sem_up(&s)  
9 }
```

The current execution state of the machine is that thread T_1 is about to call the function in line 7, and Thread T_2 is about to execute line 6.

(For any information not given in the question, you should make reasonable assumptions and justify them in your answer.)

- a Describe what state the scheduler must maintain for the semaphore s .
- b Briefly explain the semantics of the three semaphore API calls used above.
- c State, and briefly justify, what the value X in line 3 must be.
- d Write down a possible complete execution trace for T_1 and T_2 . You should list (1) the executed source code lines by the threads, (2) the thread stack, (3) the semaphore state, and (4) the blocking/unblocking/termination of threads.
- e Write down, using source code that resembles C, the implementation that the OS kernel provides for (1) the scheduler, and (2) the two functions, `sem_down(...)` and `sem_up(...)`.

(Your source code does not have to be complete or syntactically correct. You may assume the implementation of reasonable helper functions.)

The five parts carry, respectively, 10%, 15%, 10%, 30%, and 35% of the marks.

- 2a We have an architecture with 16-bit addresses and virtual memory support using a single-level page table. Each page is 2 KBytes, and each page table entry contains the frame's physical address on the most significant bits; the least significant bit represents the *valid* bit; the second least significant bit is the *modified* (dirty) bit.

The current page table is described with the following entries, using hexadecimal notation:

0: 0x4A00
1: 0x6C01
2: 0x4001
3: 0x3803
4: 0x3201
5: 0x3F01
6...N: 0x0000

- i) How many entries should the page table have to translate all possible virtual addresses? (i.e., what is the value of N above)
 - ii) Briefly describe how we should change the page table entries above, to detect null pointer dereferences by ensuring that accesses to the first 4 KBytes of virtual addresses always fail.
 - iii) Assume that we have a fully associative TLB with two entries, with pure LRU replacement, and the following list of virtual memory accesses:
0x1111, 0x17B9, 0x1F6D, 0x2A02, 0x1800, 0x13
Calculate the total number of TLB hits, as well as the number of memory accesses to the page table that the MMU performs. Assume that the TLB starts empty.
 - iv) We want to page out frame 7. List the entries in the page table that we must invalidate to page this frame out. How many memory accesses to the page table do we need to find the affected pages?
 - v) Briefly describe how we can make the page out above faster, by reducing the number of memory accesses to the page table.
- b We have an application that needs to process a large amount of data, that runs in a system with four spinning disks that have the same hardware configuration.
- i) If we know that the application will mostly perform random reads, can we optimize the application performance by using RAID level 1 (mirroring) or RAID level 5? Briefly justify your answer.

- ii) If we know that the application will mostly perform random writes, can we optimize the application performance by using RAID level 1 (mirroring) or RAID level 5? Briefly justify your answer.
 - iii) For the mostly-write application, should we use unbuffered or buffered I/O to improve performance? Briefly justify your answer.
- c We have an inode-based file system, where every inode contains 6 direct pointers, 1 pointer to a (single) indirect block, and 1 pointer to a doubly indirect block. Each pointer is 4 Bytes long, every disk block is 1024 Bytes, and each indirect block fills an entire disk block.
- i) Which of the four I/O layers do we have to modify (user-level I/O software, device-independent OS software, device drivers, and interrupt handlers), in order to reduce the number of indirect pointers?
 - ii) Which of the four I/O layers do we have to modify (user-level I/O software, device-independent OS software, device drivers, and interrupt handlers), if we increase the disk block size?
 - iii) How many blocks does a softlink occupy in this file system?

The three parts carry, respectively, 50%, 30%, and 20% of the marks.